Kingdom's Last Stand: Fantasy Medieval Tower Defense

1. Project Overview

*This project is a **Fantasy Medieval Tower Defense** game developed using Python and Pygame. The game places the player in the role of a kingdom defender, tasked with protecting their castle from waves of medieval enemies such as goblins, orcs, and dark knights.*

The core mechanic revolves around strategically placing and upgrading defensive towers along a predefined path. Each tower type has unique attributes including attack range, damage, and attack speed. Players must manage limited gold resources earned by defeating enemies to build and upgrade towers before each wave begins.

The game features multiple tower types, diverse enemy waves with increasing difficulty, a resource management system, and a real-time statistics tracking system that records gameplay data such as enemies defeated, damage dealt, gold spent, towers placed, and survival time. All collected data is visualized through graphs and charts at the end of each game session.

2. Project Review

Existing Project Reference: Bloons Tower Defense (Ninja Kiwi)

Bloons Tower Defense (BTD) is one of the most well-known tower defense games, featuring path-based enemy movement, multiple tower types with upgrade trees, and progressively difficult waves. The game serves as a

strong reference for core tower defense mechanics including tower placement, resource management, and enemy pathing.

Key Improvements in This Project:

While BTD provides an excellent foundation, this project introduces the following enhancements:

1. ***Statistical Data Collection*** *— Unlike BTD, this game actively records and visualizes gameplay statistics such as damage dealt per tower, enemies defeated per wave, gold efficiency, and survival time. This transforms the game into an analytical tool as well as an entertainment experience.*
2. ***OOP Architecture*** *— The game is built with a clean object-oriented design with clearly defined classes, making the codebase more maintainable and expandable compared to typical student projects.*
3. ***Fantasy Medieval Theme with Original Assets*** *— The game features an original themed environment with custom-designed maps and enemy types tailored to a medieval fantasy setting.*
4. ***Simplified Scope for Completeness*** *— Rather than overwhelming complexity, this project focuses on delivering a polished, bug-free experience with core mechanics fully implemented.*

3. Programming Development

3.1 Game Concept

"Kingdom's Last Stand" is a Fantasy Medieval Tower Defense game built with Python and Pygame. The player's objective is to prevent waves of medieval enemies — including goblins, orcs, and dark knights — from reaching and destroying the kingdom's castle at the end of a predefined path.

Core Mechanics:

- *Players begin each round with a fixed amount of gold*
- *Towers are placed on designated tiles alongside the enemy path*

- *Enemies spawn in waves and follow the path toward the castle*
- *Towers automatically detect and attack enemies within their range*
- *Defeating enemies rewards the player with gold to build or upgrade more towers*
- *The game ends when the castle's HP reaches zero, or all waves are cleared*

Key Features / Selling Points:

- *3 unique tower types: Archer Tower, Mage Tower, and Cannon Tower — each with different range, damage, and attack speed*
- *3 enemy types with different HP and movement speed*
- *Tower upgrade system (Level 1 → Level 2)*
- *Wave progression system with increasing difficulty*
- *Real-time HUD showing gold, castle HP, and current wave*
- *End-game statistics screen with data visualization*

3.2 Object-Oriented Programming Implementation

The game is implemented using the following classes:

Class 1: Game

- **Role:** Main controller that manages the overall game loop, state transitions, and coordinates all other components
- **Key Attributes:** `screen`, `clock`, `current_wave`, `gold`, `castle_hp`, `towers[]`, `enemies[]`
- **Key Methods:** `run()`, `update()`, `draw()`, `next_wave()`, `check_game_over()`

`Class 2: Tower (Base Class)`

- **Role:** Abstract base class for all tower types. Defines shared attributes and methods that all towers inherit
- **Key Attributes:** `level, damage, attack_range, attack_speed, position, cost`
- **Key Methods:** `find_target(), attack(), upgrade(), draw()`

Class 2a: ArcherTower(Tower)

- **Role:** Inherits from Tower. Fast attack speed, low damage, long range. Specializes in targeting fast enemies
- **Key Attributes:** `damage = 25, attack_range = 150, attack_speed = 1.5`

Class 2b: MageTower(Tower)

- **Role:** Inherits from Tower. Slow attack speed, high damage, medium range. Effective against high HP enemies
- **Key Attributes:** `damage = 60, attack_range = 120, attack_speed = 0.8`

Class 2c: CannonTower(Tower)

- **Role:** Inherits from Tower. Deals splash damage in an area, effective against groups of enemies
- **Key Attributes:** `damage = 40, attack_range = 100, attack_speed = 0.5, splash_radius = 50`
-

Class 3: Enemy

- **Role:** Represents an enemy unit that follows the predefined path toward the castle
- **Key Attributes:** `enemy_type`, `hp`, `max_hp`, `speed`, `reward_gold`, `path_index`, `position`
- **Key Methods:** `move()`, `take_damage()`, `is_dead()`, `draw()`

Class 4: Wave

- **Role:** Manages enemy spawning logic, controls the sequence and timing of enemy waves
- **Key Attributes:** `wave_number`, `enemy_list[]`, `spawn_interval`, `spawn_timer`, `is_complete`
- **Key Methods:** `spawn_next()`, `update()`, `is_wave_complete()`

Class 5: StatsTracker

- **Role:** Records and stores all gameplay statistics data for analysis and visualization
- **Key Attributes:** `enemies_defeated`, `damage_dealt[]`, `gold_spent[]`, `towers_placed[]`, `survival_time`
- **Key Methods:** `record_kill()`, `record_damage()`, `record_gold_spent()`, `save_to_csv()`, `generate_report()`

Class 6: UIManager

- **Role:** Handles all on-screen UI elements including HUD, tower selection panel, and end-game statistics screen

- **Key Attributes:** `font`, `hud_elements[]`, `tower_panel`, `stats_screen`
- **Key Methods:** `draw_hud()`, `draw_tower_panel()`, `draw_stats_screen()`

3.3 Algorithms Involved

1. Pathfinding — Waypoint-Based Path Following Enemies follow a predefined path using a waypoint system. Each enemy stores a list of waypoints (x, y coordinates) and moves toward the next waypoint until reaching the castle. This approach is simple, efficient, and suitable for a fixed-map tower defense game.

2. Collision Detection — Radius-Based Range Checking Towers detect enemies within their attack range using circular collision detection. The Euclidean distance formula is calculated between the tower's position and each enemy's position. If the distance is less than or equal to the tower's range, the enemy is considered a valid target.

3. Target Selection — Priority-Based Targeting When multiple enemies are within a tower's range, the tower selects the enemy that has traveled the furthest along the path (highest path index). This mimics real tower defense behavior where towers prioritize the most dangerous enemy closest to the castle.

4. Wave Difficulty Scaling — Rule-Based Logic Each wave increases in difficulty using a rule-based scaling system. Enemy HP, speed, and quantity increase proportionally with the wave number, creating a progressively challenging experience.

5. Event-Driven Mechanics The game uses Pygame's event system to handle player interactions such as tower placement, tower selection, and wave initiation. All user inputs are processed through an event loop, keeping game logic and input handling cleanly separated.

4. Statistical Data (Prop Stats)

4.1 Data Features

The following five features will be tracked during gameplay. Each feature is recorded every game session and will accumulate more than 100 rows of data through repeated playthroughs.

Feature 1: Enemies Defeated per Wave *Records the number of enemies successfully defeated in each wave. This tracks player performance and tower effectiveness over time.*

- *Unit: count (integer)*
- *Example record: Wave 1 → 10 enemies, Wave 2 → 15 enemies*

Feature 2: Damage Dealt per Tower *Records the total damage dealt by each tower throughout the game session. This helps analyze which tower type is most effective.*

- *Unit: damage points (integer)*
- *Example record: Archer Tower at (3,2) → 450 damage*

Feature 3: Gold Spent per Wave *Records how much gold the player spends on building or upgrading towers before each wave. This tracks player strategy and resource management behavior.*

- *Unit: gold (integer)*
- *Example record: Wave 3 → 150 gold spent*

Feature 4: Castle HP Remaining per Wave *Records the castle's remaining HP at the end of each wave. This measures how well the player defended against each wave.*

- *Unit: HP (integer)*
- *Example record: After Wave 5 → 75 HP remaining*

Feature 5: Wave Survival Time *Records the time (in seconds) taken to complete or fail each wave. This measures game pacing and player efficiency.*

- *Unit: seconds (float)*
- *Example record: Wave 4 → 38.5 seconds*

4.2 Data Recording Method

All statistical data will be recorded and stored using ***CSV files*** *managed through Python's built-in* `csv` *module. This approach requires no external database setup and is simple to implement within Pygame.*

The recording system works as follows:

- *A single* `game_stats.csv` *file is maintained throughout all game sessions*
- *Each new session appends new rows to the existing file without overwriting previous data*
- *The* `StatsTracker` *class handles all data recording throughout the gameplay*
- *Data is written to the CSV file automatically at the end of each wave and at game over*
- *Each row in the CSV represents one recorded event during gameplay*

The CSV file will contain the following columns:

Column	*Description*
`session_id`	*Unique identifier for each game session*
`wave_number`	*The wave number when data was recorded*
`enemies_defeated`	*Number of enemies defeated in that wave*
`damage_dealt`	*Total damage dealt by all towers in that wave*
`gold_spent`	*Gold spent by the player in that wave*
`castle_hp`	*Castle HP remaining after the wave*
`survival_time`	*Time taken to complete the wave (seconds)*

4.3 Data Analysis Report

The recorded CSV data will be analyzed and visualized using Python's `matplotlib` *and* `pandas` *libraries. The analysis will be presented at the end of each game session through an in-game statistics screen, as well as exported as a report.*

Statistical Measures Used:

- ***Mean (Average)*** *— average enemies defeated per wave, average gold spent per wave, average survival time per wave*
- ***Maximum / Minimum*** *— highest and lowest values of enemies defeated, damage dealt, and castle HP remaining across all waves*
- ***Sum*** *— total damage dealt, total gold spent across the entire session*

- ***Trend Analysis*** *— how castle HP remaining and enemies defeated change across waves, measured by tracking the exact integer value of each metric at the end of every wave*
- *This is our mock histogram data.*
- 

Visualizations:

4.1 Data Features

Feature	*Why it is good to have this data? What can it be used for?*	*How will you obtain 100 values of this feature data?*	*Which variable (and which class will you collect this from)?*	*How will you display this feature data?*
Enemies Defeated per Wave	*Measures player effectiveness and tower performance. Used to analyze difficulty progression across waves.*	*Recorded automatically at the end of every wave. 10 waves × 10+ sessions = 100+ records.*	*`enemies_defeated` from `StatsTracker`*	*Histogram + summary statistics table*
Damage Dealt per Wave	*Measures total offensive output per wave. Used to evaluate which tower configuration is most effective.*	*Recorded at the end of every wave across multiple sessions.*	*`damage_dealt` from `StatsTracker`*	*Histogram + Line graph*
Gold Spent per Wave	*Tracks player resource management behavior. Used to analyze spending strategy across waves.*	*Recorded every wave when player builds or upgrades towers.*	*`gold_spent` from `StatsTracker`*	*Histogram + Bar graph*
Castle HP Remaining per Wave	*Measures how well the player defended each wave. Tracks survival performance over time.*	*Recorded automatically at the end of every wave.*	*`castle_hp` from `StatsTracker`*	*Line graph + summary statistics table*
Wave Survival Time	*Measures how long each wave lasts. Used to analyze game pacing and player efficiency.*	*Recorded automatically when each wave starts and ends.*	*`survival_time` from `StatsTracker`*	*Histogram + summary statistics table*

4.2 Data Recording Method

All statistical data will be recorded and stored in a single CSV file named `game_stats.csv`, managed through Python's built-in `csv` module. Data from every game session is appended to this file, allowing cumulative analysis across multiple playthroughs.

The recording system works as follows:

- *A single `game_stats.csv` file is maintained throughout all game sessions*

- *Each new session appends new rows to the existing file without overwriting previous data*
- *The `StatsTracker` class handles all data recording throughout the gameplay*
- *Data is written to the CSV file automatically at the end of each wave and at game over*
- *Each row represents one recorded event, identified by a unique `session_id` and timestamp*

Column	*Description*
`session_id`	*Unique identifier for each game session*
`wave_number`	*The wave number when data was recorded*
`enemies_defeated`	*Number of enemies defeated in that wave*
`damage_dealt`	*Total damage dealt by all towers in that wave*
`gold_spent`	*Gold spent by the player in that wave*
`castle_hp`	*Castle HP remaining after the wave*
`survival_time`	*Time taken to complete the wave (seconds)*

4.3 Data Analysis Report

Summary statistics table

Summary statistics table

Feature	Mean	Min	Max	Std Dev
Enemies Defeated per Wave	✓	✓	✓	✓
Damage Dealt per Wave	✓	✓	✓	✓
Gold Spent per Wave	✓	✓	✓	✓
Castle HP Remaining	✓	✓	✓	✓
Wave Survival Time	✓	✓	✓	✓

Graph planning table

	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	Enemies Defeated per Wave	Show distribution of enemies defeated across all recorded waves	Histogram Distribution	Enemies defeated (count range)	Frequency
Graph 2	Castle HP Remaining per Wave	Show how castle HP changes over wave progression	Line graph Time-series	Wave number	Castle HP remaining
Graph 3	Gold Spent per Wave	Show proportion of gold spending across each wave	Bar graph Proportion	Wave number	Gold spent

Categories covered: Distribution (Histogram) + Time-series (Line graph) + Proportion (Bar graph) — 3 different types, no repeats.

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Game architecture design + OOP class setup
3	10	Core game mechanics (tower placement, enemy pathing, wave system)
4	11	Tower types, enemy types, upgrade system + UI/HUD
5	12	Statistics tracking system + CSV recording
6	13	Data visualization + bug fixing + polishing
7	14	Submission week (Draft)

6. Document version

Version: 1.0
Date: 11 March 2026
Editor: Thanutdit Jiravichalert

Date	Name	Description of Revision, Feedback, Comments
14/3	Nattanan	Remove the italics in your work Working on understanding usage of inheritance When there are many towers or enemy types, use inheritance. This will reduce your work to have to manually fill the hp, speed, and sprite. This is easier to implement and makes more sense in real world. Do not separate file for the data analysis, that will make your analysis much harder Also warning, the tower defense game is proposed by many people, which can lead to comparing between 2 work. Make sure your work stands out.
21/03	Amornrit	Study inheritance vs composition in entities that behave differently (e.g., special kind of tower). Choose the approach that is suitable for your game. As P'Nattanan mentioned, you need to make your work stand out.